

Отчёт по Параллельной Реализации Стохастического Градиентного Спуска (SGD) с OpenMP

Овсянникова Софья, 414 группа

1. Обоснование выбора алгоритма

Объектом исследования является **стохастический градиентный спуск (SGD) с мини-батчами**. SGD - это итерационный оптимизационный алгоритм первого порядка, широко используемый для обучения моделей машинного.

Цель работы - оценить эффективность параллельной реализации SGD с помощью технологии OpenMP на многоядерной архитектуре.

SGD выбран для параллельной реализации благодаря наличию высокого ресурса массового параллелизма на ключевом вычислительном этапе: вычисление частных градиентов для каждого из примеров в мини-батче полностью независимо.

Этот координатный параллелизм позволяет распределить основную вычислительную нагрузку между потоками, что гарантирует значительный выигрыш в скорости.

Основное ограничение - последовательная зависимость между шагами обновления весов, что не позволяет ускорить весь процесс по закону Амдала. Мы фокусируемся на ускорении самого затратного шага - вычислении градиента на батче.

2. Ссылка на описание в энциклопедии AlgoWiki

https://algowiki-project.org/ru/Участница:OvsyannikovaSA/Стохастический_градиентный_спуск

https://algowiki-project.org/ru/Участница:OvsyannikovaSA/Программная_реализация_стохастического_градиентного_спуска

3. Программно-аппаратная среда и методика экспериментов

Эксперименты проведены на суперкомпьютере «Ломоносов-2».

Технология параллелизма: OpenMP

Компилятор: gcc/14.1

Опции компиляции: -std=c++17 -O3 -fopenmp

4. Методика Проведения Экспериментов

Рассматривается задача линейной регрессии.

Размер задачи фиксирован: общее число примеров N и количество признаков фиксировано;

Размер батча B : 256, 512, 1024, 2048, 4096, 8192;

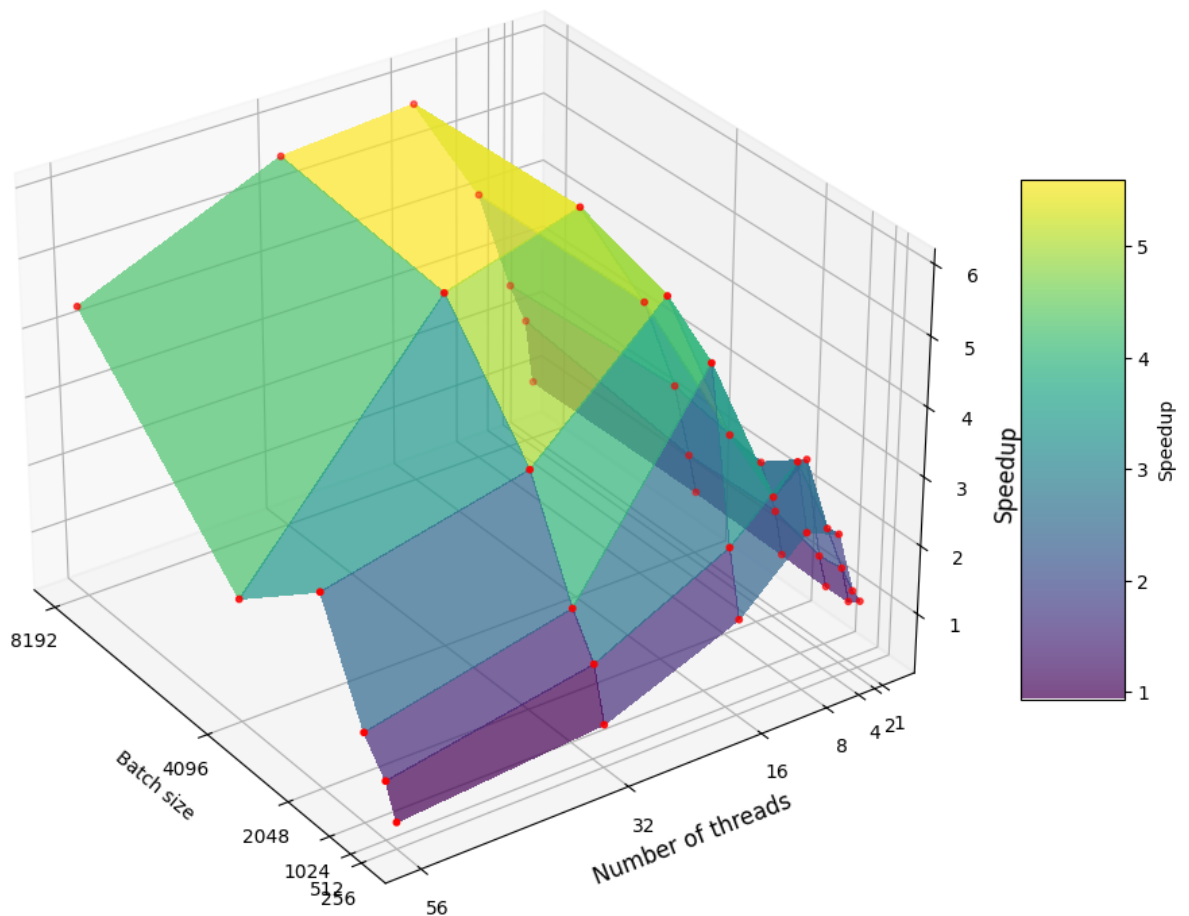
Число потоков P : 1, 2, 4, 8, 16, 32, 56;

Метрика: ускорение, определяемое как отношение времени выполнения последовательной версии к времени выполнения параллельной версии.

5. Результаты экспериментов и анализ масштабируемости

На рисунке представлена 3D-поверхность ускорения, полученная методом интерполяции по измеренным точкам. На графике ось Ox - размер батча B , ось Oy - число потоков, ось Oz - ускорение.

3D-surface, speedup (interpolation)



Объяснение полученных результатов

Наблюдаемая поверхность ускорения демонстрирует ожидаемое поведение алгоритма SGD:

1. **Зависимость от числа потоков:** при фиксированном большом размере батча (например, $B=8192$) наблюдается линейный рост ускорения до примерно $P=16$ или $P=32$. Это подтверждает наличие высокого ресурса массового параллелизма на этапе вычисления градиентов. При дальнейшем увеличении числа потоков (от $P=32$ до $P=56$) ускорение насыщается или начинает падать. Накладные расходы на создание, синхронизацию потоков и время, затрачиваемое на глобальную операцию редукции, начинают доминировать над временем полезных вычислений.
2. **Зависимость от размера батча:** самые высокие значения ускорения наблюдаются для максимальных размеров батчей ($B=8192$). Это следствие того, что с ростом B растет и объем параллельной работы в рамках одного шага SGD. Увеличение B позволяет процессорам работать дольше без необходимости синхронизации, что снижает относительную долю накладных расходов. Для малых батчей (например, $B=256$) ускорение низкое даже при большом P . Это объясняется тем, что время, необходимое на синхронизацию потоков OpenMP, становится сравнимо с временем самого вычисления градиента.

6. Выводы

SGD обладает хорошей сильной масштабируемостью на многоядерной архитектуре с общей памятью (OpenMP), но только при условии, что размер батча достаточно велик.

Предельная масштабируемость ($S=6.0$ при $P=56$) значительно ниже идеальной ($S=56$). Это обусловлено последовательной фазой редукции и накладными расходами на потоки, которые становятся доминирующими при $P > 32$.

7. Комментарии к программной реализации

Основной код находится в файле `sgd.cpp`.

Файл `run_exps.sh` содержит команды `bash` для перебора параметров SGD (числа потоков и размера батчей).

Файл `collect_speedup.sh` содержит команды `bash` для извлечения требуемой информации из лог-файлов и записи в файл `speedup_data.csv`.

Файл `plot_speedup.py` строит график ускорения по полученной таблице.

Архивы `results.zip` и `losses.zip` содержат результаты работы алгоритма для соответствующих наборов параметров.